

Project Stage 1: language design, lexer and parser

Languages and Compilers Design · Course project, stage 1 of 3 · deadline 6 October 2026, 23:59

Complete the first stage of the course project: design a programming language of your own and implement its lexer and its parser, which builds an Abstract Syntax Tree (AST). The stage stands on what the practices gave you — the environment, a state-machine lexer, a recursive-descent parser and a tree — and applies it to a language that is yours.

Grading (15 points total)

Submission format and code style
Language design and EBNF description
Lexer and parser implementation
Test cases
README.md with instructions

2 pt
4 pt
5 pt
3 pt
1 pt

`ai_usage.txt` is required: a missing or incomplete file is an automatic grade of 0 for the stage. The lexer and the parser are written by hand: a lexer built on regular expressions, `split()` or a lexer generator, or a parser from a parser generator, gets 0 for the implementation. Late submission is 0.

TASK 1 — AI USAGE DISCLOSURE (REQUIRED)

Include a file named `ai_usage.txt` in your submission. It must contain a full description of your AI usage, including links to the chat transcripts or the prompts as you sent them, what you took as it was and what you changed. If you did not use AI, say so in the file.

TASK 2 — DESIGN YOUR LANGUAGE (4 POINTS)

Propose the syntax and the rules of your own programming language and describe it in EBNF. To add creativity:

- use emojis for operators, type names or other keywords;
- introduce unique syntax elements: one symbol per line, inverted lines, or any other feature you can lex and parse.

Your language must support at least:

- two integer types — the equivalents of `i32` and `i64`;
- a boolean type;
- constants, mutable variables and constant (immutable) variables;
- assignment;
- comparison operators — at least the equivalents of `==` and `!=`;
- type overflow checks;
- `if / else`, equivalent to `"if" boolExpr "{" stmt+ "}" [ "else" "{" stmt+ "}" ]`: the `if` block and the `else` block contain at least one statement each, and the `else` clause is optional.

The EBNF is complete: every construct your parser accepts is in the grammar, and nothing else is. Put it in `grammar.ebnf`.

An emoji is several bytes. Your lexer reads bytes, so a keyword written as an emoji is a byte sequence for the state machine — the same mechanism as a keyword written in letters.

TASK 3 — OPTIONAL: VS CODE EXTENSION FOR YOUR LANGUAGE

To make emojis and other symbols easier to type, create a simple VS Code extension similar to `VSCoDeKaguLang-Support`. It should treat your emojis as keywords and offer completion or replacement. No points; link it from the README if you make one.

TASK 4 — IMPLEMENT THE LEXER AND THE PARSER (5 POINTS)

- The **lexer** scans the source and produces tokens with a hand-written state machine over bytes. Every token carries its kind, text, line and column; lexical errors are reported with `line:column`.
- The **parser** takes the tokens as input and builds the AST by recursive descent: `peek` and `eat` over the token stream, one function per grammar rule.
- Use a **class hierarchy** for the AST nodes: `ASTNode`, `ProgramNode`, `StmtNode` with its statement classes, `ExprNode` with its expression classes, and so on. Every node keeps the position of the token it came from.

- The program **outputs a description of the AST**, including the flags and the fields of each node (mutable or not, which type, which operator), indented by depth: `python3 compiler.py --ast input.txt`. Your tests compare against this output, so keep it stable.
- A syntax error is one line on stderr, `compilation error: line L:C: message`, a non-zero exit code and no output.

Semantic checks and code generation for your language belong to the next project stage.

Warning. Grammar first, node hierarchy second, parser third. A node class that changes every time a grammar rule changes is a sign the grammar was not finished.

TASK 5 — WRITE TEST CASES (3 POINTS)

- at least **15 test cases for the happy path**: valid programs that parse and produce the expected AST, each with a file that holds the expected dump;
- at least **15 test cases for error scenarios**: invalid programs that must cause a lexical or a syntax error, each with a file that holds the expected error line;
- a script that runs all of them and says which ones failed.

Name a test after what it covers: `if_else_nested.txt`, `unclosed_block.txt`. Every feature of Task 2 is covered from both sides: a program that uses it and a program that misuses it.

TASK 6 — README.MD WITH INSTRUCTIONS (1 POINT)

`README.md` in the repository contains:

- a description of your language: its syntax and its features, with a short example for each;
- the steps to build and run your compiler;
- how to run the tests.

SUBMISSION

1. Create a **new repository** for the course project (GitHub or GitLab, private or public) and add vovaskochko as a collaborator on the first day — a reviewer can be requested only after the invitation is accepted.
2. Track your progress on a separate branch named `stage1`, with real commits and real messages.
3. When the stage is complete, open a pull request from `stage1` into `main` and add vovaskochko as a reviewer. Do not merge it before the review.
4. Make sure the pull request contains everything: the source code, `grammar.ebnf`, `ai_usage.txt`, `README.md`, the tests and the script that runs them.
5. Submit in Moodle: the link to the repository, the link to the pull request, and `ai_usage.txt`. The last commit of the pull request before the deadline is what is graded.